

EEE3023 Error Correcting Computing Lab Report

A lower bit-error rate is more desirable as this means that there are fewer bit-errors per transmitted bit.

Results

Repetition Codes

Repetition codes will repeat the message bit n times to encode the message. Majority decoding is used to retrieve the message, where more 1's will return a 1 and more 0's will return a 0.

```
1 %Experiment 1: Repetition Code
2
3 clear();
4 tic();
5
6 K=1; %message length
7 N=3; %codeword length
8 D=3; %minimum Hamming distance
9 Rc=N/K; %code rate
10 message=zeros(1,K); %initialise k-bit message to all zeros, 1 by k matrix
11 codeword=zeros(1,N); %initialise n-bit codeword to all zeros
12 snr=[0 1 2 3 4 5 6 7 8 9 10]; %range of signal-to-noise ratios in dB to test.
13 frames=[1000 1000 10000 100000 1000000 1000000 1000000 1000000 1000000 1000000 1000000]; %number of codewords generated for each SNR
14 snr_uncoded=lininspace(0,max(snr),length(snr)); %range of signal-to-noise ratios for BER of uncoded BPSK which we will compare our results with.
15 %y = linspace(x1, x2, n) generates n points between x1 and x2.
16 %Evenly spaced intervals. Length is a built in function for number of
17 %elements in an array
18 Eb_No=10.^(snr/10); %convert snr in dB to Eb/No
19 ber=zeros(1,length(snr)); %initialise vector of bit-error rates for each snr value
20 ber_soft=zeros(1,length(snr)); %initialise vector of bit-error rates (soft decoding)
21
22 G=ones(1,N); %Repetition code generator matrix
23 %We have set the message and codeword lengths, hamming distance and code
24 %rate, SNR from dB to linear.
25 %We initialise arrays for message and codewords, bit error rate(ber) and a
26 %generator matrix.
27
28 for s=1:length(snr) %going through each element of snr array
29     errors=0; %initialise error counter at the start of each snr
30     errors_soft=0; %initialise soft error counter at the start of each snr
```

```

12 sigma=sqrt(1/(2*Nc*Eb_H0(s))); %Standard deviation of noise
13 for f=1:frames(x) %loop for frame values for the current Eb value
14     message=randi([0 1],1,K); %Generate 1-bit message with random 0s and 1s for message form
15     index=randi([0 2],1,K); %Multiply message by generate matrix to obtain the n-bit repetition codeword, 2 represents modulo 2 which is for bit arithmetic.. (mod sets the base).
16     %Now we map the 1's and 0's to phase or not of phase.
17     w=2*cos(2*pi*f); %BPSK signal obtained by mapping a bit 0 to +1 (in-phase carrier signal) and a bit 1 to -1 (out-of-phase carrier signal)
18     % If it's 0 then 1-0 = +1 and if it's a 1 then 1-2 = -1.
19
20     r=sigma*randn(1,length(x)); %Generate white Gaussian noise, randn generates random numbers from a normal distribution of mean 0 and variance of 1.
21     % 1, length(x) generates 1 row and length(x) columns
22     y=w+r; %Add noise to BPSK signal y is now the signal
23
24     d=y(0); %BPSK demodulation. If the received signal is negative then the demodulated bit is 1, else the demodulated bit is 0.
25
26     %Majority Decoding
27
28     n1=sum(d); %Count the number of 1s in the received binary word d, sum is a function for number of nonzero elements.
29     n0=n-1; %The number of 0s in the received binary word d
30
31     if n1>N/2 %If the number of 1s is greater than the number 0s in d
32         decoded_message=1; %Decoded message bit is 1
33     else
34         decoded_message=0; %Since the decoded message bit is 0
35     end
36
37     %Soft Majority Decoding
38     %Sum the received samples rather than count the number of elements that are non-zero. If sum(y) < 0 then decide bit 1, else bit 0.
39     decoded_message_soft = (sum(y) < 0);
40
41     %If decoded_message=message %Hard decision error counter
42     errors=errors+1;
43 end
44
45 %If decoded_message_soft=message %Soft decision error counter
46 errors_soft=errors_soft+1;
47 end
48
49 %
50 %
51 %
52 %
53 %
54 %
55 %
56 %
57 %
58 %
59 %
60 %
61 %
62 %
63 %
64 %
65 %
66 %
67 %
68 %
69 %
70 %
71 %
72 %
73 %
74 %
75 %
76 %
77 %
78 %
79 %
80 %
81 %
82 %
83 %
84 %
85 %
86 %
87 %
88 %
89 %
90 %
91 %
92 %
93 %
94 %
95 %
96 %
97 %
98 %
99 %
100 %
101 %
102 %
103 %
104 %
105 %
106 %
107 %
108 %
109 %
110 %
111 %
112 %
113 %
114 %
115 %
116 %
117 %
118 %
119 %
120 %
121 %
122 %
123 %
124 %
125 %
126 %
127 %
128 %
129 %
130 %
131 %
132 %
133 %
134 %
135 %
136 %
137 %
138 %
139 %
140 %
141 %
142 %
143 %
144 %
145 %
146 %
147 %
148 %
149 %
150 %
151 %
152 %
153 %
154 %
155 %
156 %
157 %
158 %
159 %
160 %
161 %
162 %
163 %
164 %
165 %
166 %
167 %
168 %
169 %
170 %
171 %
172 %
173 %
174 %
175 %
176 %
177 %
178 %
179 %
180 %
181 %
182 %
183 %
184 %
185 %
186 %
187 %
188 %
189 %
190 %
191 %
192 %
193 %
194 %
195 %
196 %
197 %
198 %
199 %
200 %
201 %
202 %
203 %
204 %
205 %
206 %
207 %
208 %
209 %
210 %
211 %
212 %
213 %
214 %
215 %
216 %
217 %
218 %
219 %
220 %
221 %
222 %
223 %
224 %
225 %
226 %
227 %
228 %
229 %
230 %
231 %
232 %
233 %
234 %
235 %
236 %
237 %
238 %
239 %
240 %
241 %
242 %
243 %
244 %
245 %
246 %
247 %
248 %
249 %
250 %
251 %
252 %
253 %
254 %
255 %
256 %
257 %
258 %
259 %
260 %
261 %
262 %
263 %
264 %
265 %
266 %
267 %
268 %
269 %
270 %
271 %
272 %
273 %
274 %
275 %
276 %
277 %
278 %
279 %
280 %
281 %
282 %
283 %
284 %
285 %
286 %
287 %
288 %
289 %
290 %
291 %
292 %
293 %
294 %
295 %
296 %
297 %
298 %
299 %
300 %
301 %
302 %
303 %
304 %
305 %
306 %
307 %
308 %
309 %
310 %
311 %
312 %
313 %
314 %
315 %
316 %
317 %
318 %
319 %
320 %
321 %
322 %
323 %
324 %
325 %
326 %
327 %
328 %
329 %
330 %
331 %
332 %
333 %
334 %
335 %
336 %
337 %
338 %
339 %
340 %
341 %
342 %
343 %
344 %
345 %
346 %
347 %
348 %
349 %
350 %
351 %
352 %
353 %
354 %
355 %
356 %
357 %
358 %
359 %
360 %
361 %
362 %
363 %
364 %
365 %
366 %
367 %
368 %
369 %
370 %
371 %
372 %
373 %
374 %
375 %
376 %
377 %
378 %
379 %
380 %
381 %
382 %
383 %
384 %
385 %
386 %
387 %
388 %
389 %
390 %
391 %
392 %
393 %
394 %
395 %
396 %
397 %
398 %
399 %
400 %
401 %
402 %
403 %
404 %
405 %
406 %
407 %
408 %
409 %
410 %
411 %
412 %
413 %
414 %
415 %
416 %
417 %
418 %
419 %
420 %
421 %
422 %
423 %
424 %
425 %
426 %
427 %
428 %
429 %
430 %
431 %
432 %
433 %
434 %
435 %
436 %
437 %
438 %
439 %
440 %
441 %
442 %
443 %
444 %
445 %
446 %
447 %
448 %
449 %
450 %
451 %
452 %
453 %
454 %
455 %
456 %
457 %
458 %
459 %
460 %
461 %
462 %
463 %
464 %
465 %
466 %
467 %
468 %
469 %
470 %
471 %
472 %
473 %
474 %
475 %
476 %
477 %
478 %
479 %
480 %
481 %
482 %
483 %
484 %
485 %
486 %
487 %
488 %
489 %
490 %
491 %
492 %
493 %
494 %
495 %
496 %
497 %
498 %
499 %
500 %
501 %
502 %
503 %
504 %
505 %
506 %
507 %
508 %
509 %
510 %
511 %
512 %
513 %
514 %
515 %
516 %
517 %
518 %
519 %
520 %
521 %
522 %
523 %
524 %
525 %
526 %
527 %
528 %
529 %
530 %
531 %
532 %
533 %
534 %
535 %
536 %
537 %
538 %
539 %
540 %
541 %
542 %
543 %
544 %
545 %
546 %
547 %
548 %
549 %
550 %
551 %
552 %
553 %
554 %
555 %
556 %
557 %
558 %
559 %
560 %
561 %
562 %
563 %
564 %
565 %
566 %
567 %
568 %
569 %
570 %
571 %
572 %
573 %
574 %
575 %
576 %
577 %
578 %
579 %
580 %
581 %
582 %
583 %
584 %
585 %
586 %
587 %
588 %
589 %
590 %
591 %
592 %
593 %
594 %
595 %
596 %
597 %
598 %
599 %
600 %
601 %
602 %
603 %
604 %
605 %
606 %
607 %
608 %
609 %
610 %
611 %
612 %
613 %
614 %
615 %
616 %
617 %
618 %
619 %
620 %
621 %
622 %
623 %
624 %
625 %
626 %
627 %
628 %
629 %
630 %
631 %
632 %
633 %
634 %
635 %
636 %
637 %
638 %
639 %
640 %
641 %
642 %
643 %
644 %
645 %
646 %
647 %
648 %
649 %
650 %
651 %
652 %
653 %
654 %
655 %
656 %
657 %
658 %
659 %
660 %
661 %
662 %
663 %
664 %
665 %
666 %
667 %
668 %
669 %
670 %
671 %
672 %
673 %
674 %
675 %
676 %
677 %
678 %
679 %
680 %
681 %
682 %
683 %
684 %
685 %
686 %
687 %
688 %
689 %
690 %
691 %
692 %
693 %
694 %
695 %
696 %
697 %
698 %
699 %
700 %
701 %
702 %
703 %
704 %
705 %
706 %
707 %
708 %
709 %
710 %
711 %
712 %
713 %
714 %
715 %
716 %
717 %
718 %
719 %
720 %
721 %
722 %
723 %
724 %
725 %
726 %
727 %
728 %
729 %
730 %
731 %
732 %
733 %
734 %
735 %
736 %
737 %
738 %
739 %
740 %
741 %
742 %
743 %
744 %
745 %
746 %
747 %
748 %
749 %
750 %
751 %
752 %
753 %
754 %
755 %
756 %
757 %
758 %
759 %
760 %
761 %
762 %
763 %
764 %
765 %
766 %
767 %
768 %
769 %
770 %
771 %
772 %
773 %
774 %
775 %
776 %
777 %
778 %
779 %
780 %
781 %
782 %
783 %
784 %
785 %
786 %
787 %
788 %
789 %
790 %
791 %
792 %
793 %
794 %
795 %
796 %
797 %
798 %
799 %
800 %
801 %
802 %
803 %
804 %
805 %
806 %
807 %
808 %
809 %
810 %
811 %
812 %
813 %
814 %
815 %
816 %
817 %
818 %
819 %
820 %
821 %
822 %
823 %
824 %
825 %
826 %
827 %
828 %
829 %
830 %
831 %
832 %
833 %
834 %
835 %
836 %
837 %
838 %
839 %
840 %
841 %
842 %
843 %
844 %
845 %
846 %
847 %
848 %
849 %
850 %
851 %
852 %
853 %
854 %
855 %
856 %
857 %
858 %
859 %
860 %
861 %
862 %
863 %
864 %
865 %
866 %
867 %
868 %
869 %
870 %
871 %
872 %
873 %
874 %
875 %
876 %
877 %
878 %
879 %
880 %
881 %
882 %
883 %
884 %
885 %
886 %
887 %
888 %
889 %
890 %
891 %
892 %
893 %
894 %
895 %
896 %
897 %
898 %
899 %
900 %
901 %
902 %
903 %
904 %
905 %
906 %
907 %
908 %
909 %
910 %
911 %
912 %
913 %
914 %
915 %
916 %
917 %
918 %
919 %
920 %
921 %
922 %
923 %
924 %
925 %
926 %
927 %
928 %
929 %
930 %
931 %
932 %
933 %
934 %
935 %
936 %
937 %
938 %
939 %
940 %
941 %
942 %
943 %
944 %
945 %
946 %
947 %
948 %
949 %
950 %
951 %
952 %
953 %
954 %
955 %
956 %
957 %
958 %
959 %
960 %
961 %
962 %
963 %
964 %
965 %
966 %
967 %
968 %
969 %
970 %
971 %
972 %
973 %
974 %
975 %
976 %
977 %
978 %
979 %
980 %
981 %
982 %
983 %
984 %
985 %
986 %
987 %
988 %
989 %
990 %
991 %
992 %
993 %
994 %
995 %
996 %
997 %
998 %
999 %
1000 %

```

Task 1:

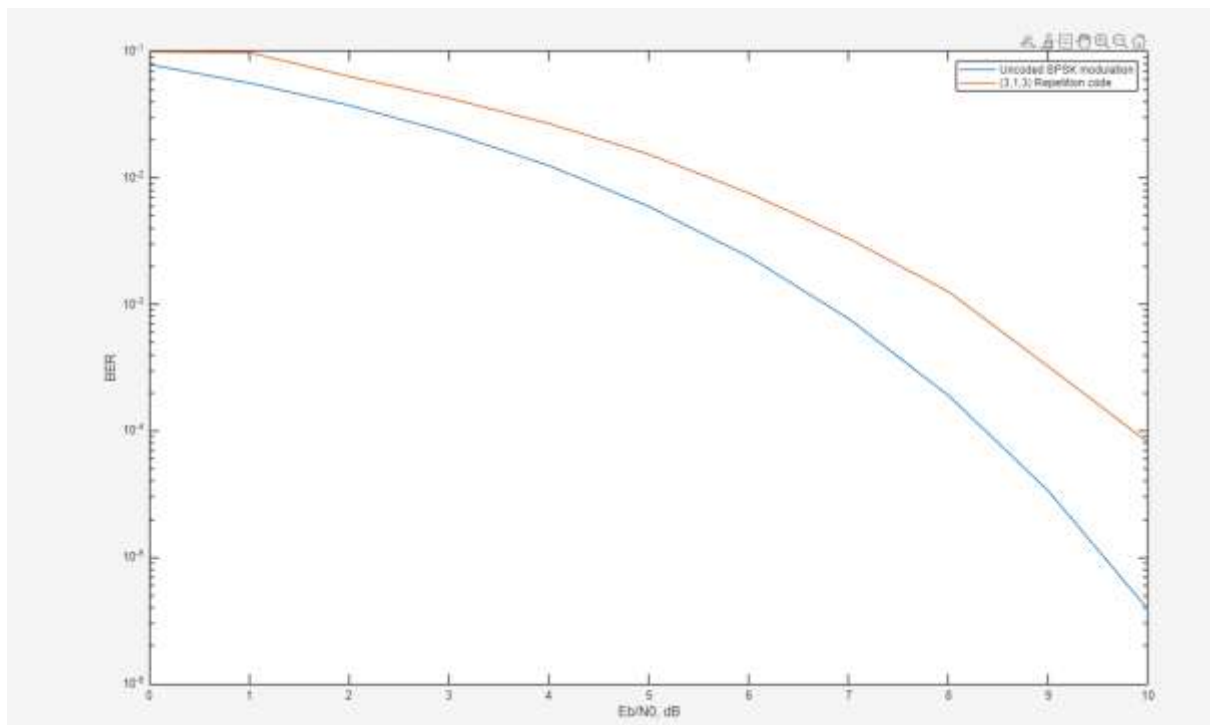


Figure 1: (3,1,3) Repetition code performance vs Uncoded BPSK modulation

Task 2:

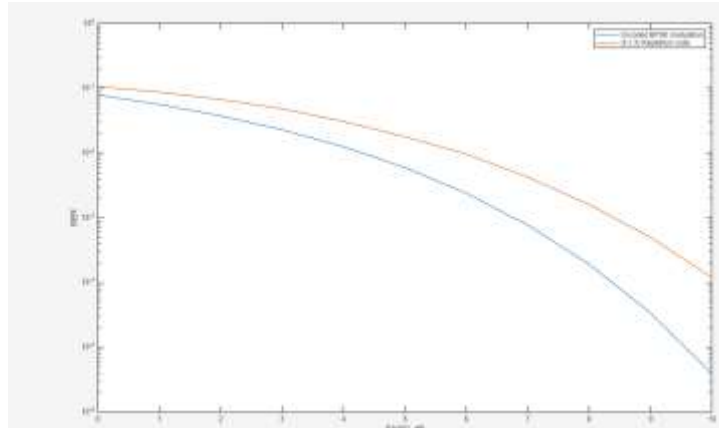


Figure 2: (5,1,5) Repetition code performance vs Uncoded BPSK modulation

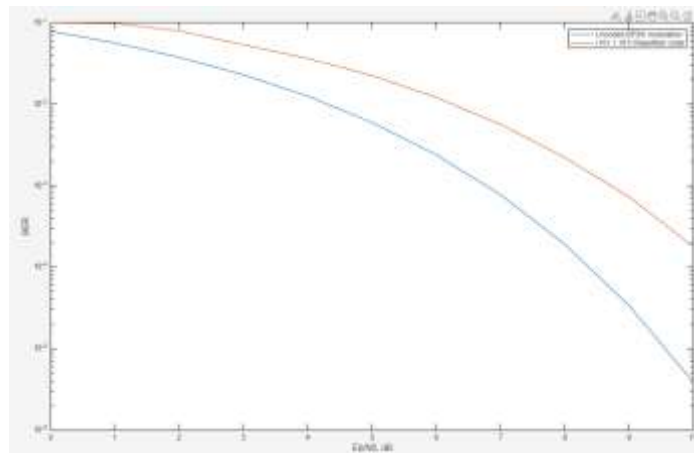


Figure 3: (101,1,101) Repetition code performance vs Uncoded BPSK modulation

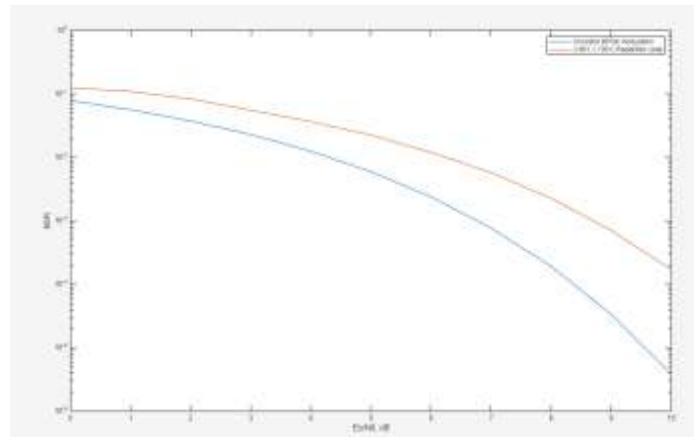


Figure 4: (1001,1,1001) Repetition code performance vs Uncoded BPSK modulation

Figure 1 shows that the (3,1,3) repetition code performance is significantly worse than Binary Phase Shift keying modulation due to having a greater bit error rate at all signal to noise ratios. They have similar bit-error rates when SNR is low since noise dominates and therefore many bit-errors will occur anyway. However, as energy dominates the additive white gaussian noise, the bit-error rate still has worse performance. This is

because more redundant bits added to the message (code rate decreases) means that each energy per bit will therefore decrease in the codeword leading to more bit-errors as each bit becomes less distinguishable to noise. Figures 2-4 show that by increasing n does not help as more repetitions lead to a greater deviation of noise (5) and more bit-errors however the trade-off is that more error corrections will also occur since majority decoding is more likely to be successful with more repetitions. This trade-off cancels out leading to similar bit-error rates per SNR value.

$$\sigma = \sqrt{\frac{1}{2R_c \times 10^{-10} \text{ SNR}}}$$

Figure 5: Formula for standard deviation of noise

Codes can error correct up to $(d-1)/2$ and detect $d-1$ errors so repetition codes have a lot of error detection capabilities however due to the sheer number of repeated bits, the transmitted bit energy is so low in the codeword.

Soft majority decoding:

Instead of using the majority bits to decode the message, the sum can be taken instead. When noise is added to the BPSK signal the summation of the values can be taken. If the summation is less than 0 then bit 1 is decided else 0. Therefore, more confident bits have a heavier weighting than bits affected more by noise therefore a lower BER. Figures

6-9 show that it is much better than the hard majority repetition codes however still not an improvement to the uncoded modulation due to the low code rates.

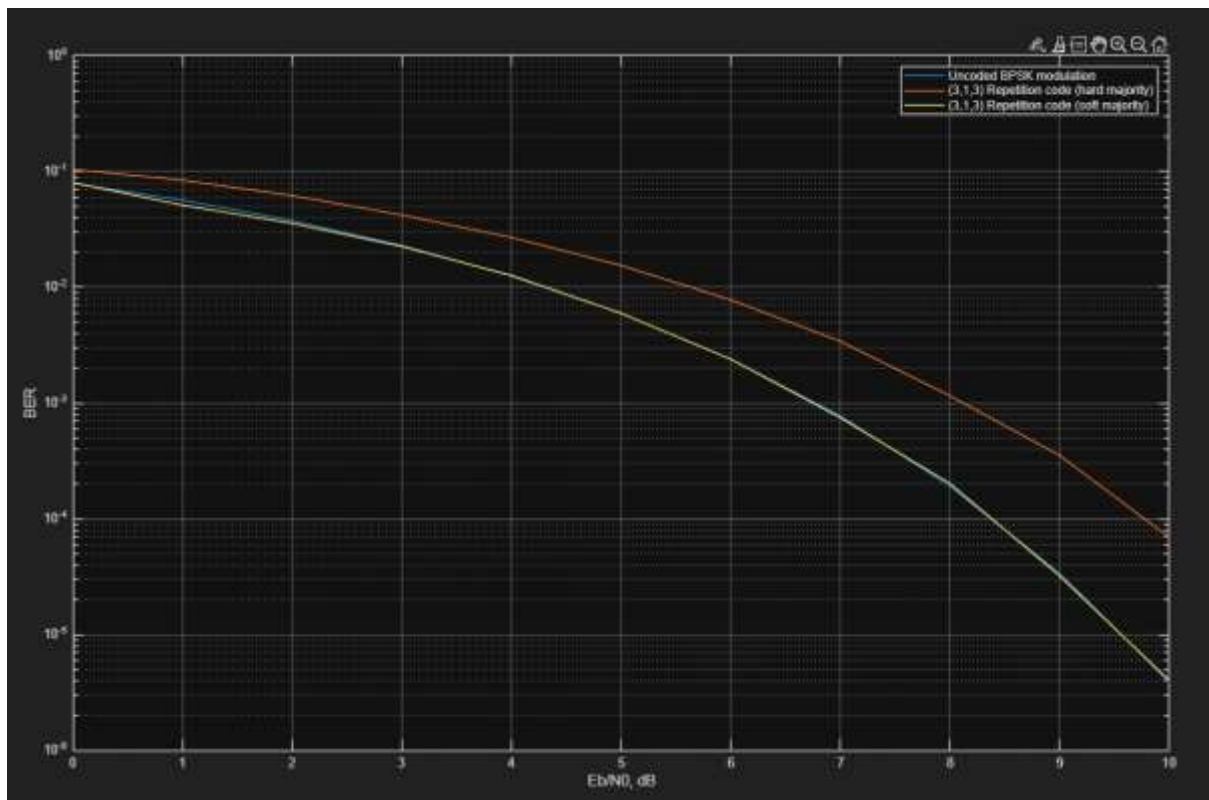


Figure 6: Soft majority decoding (3,1,3)

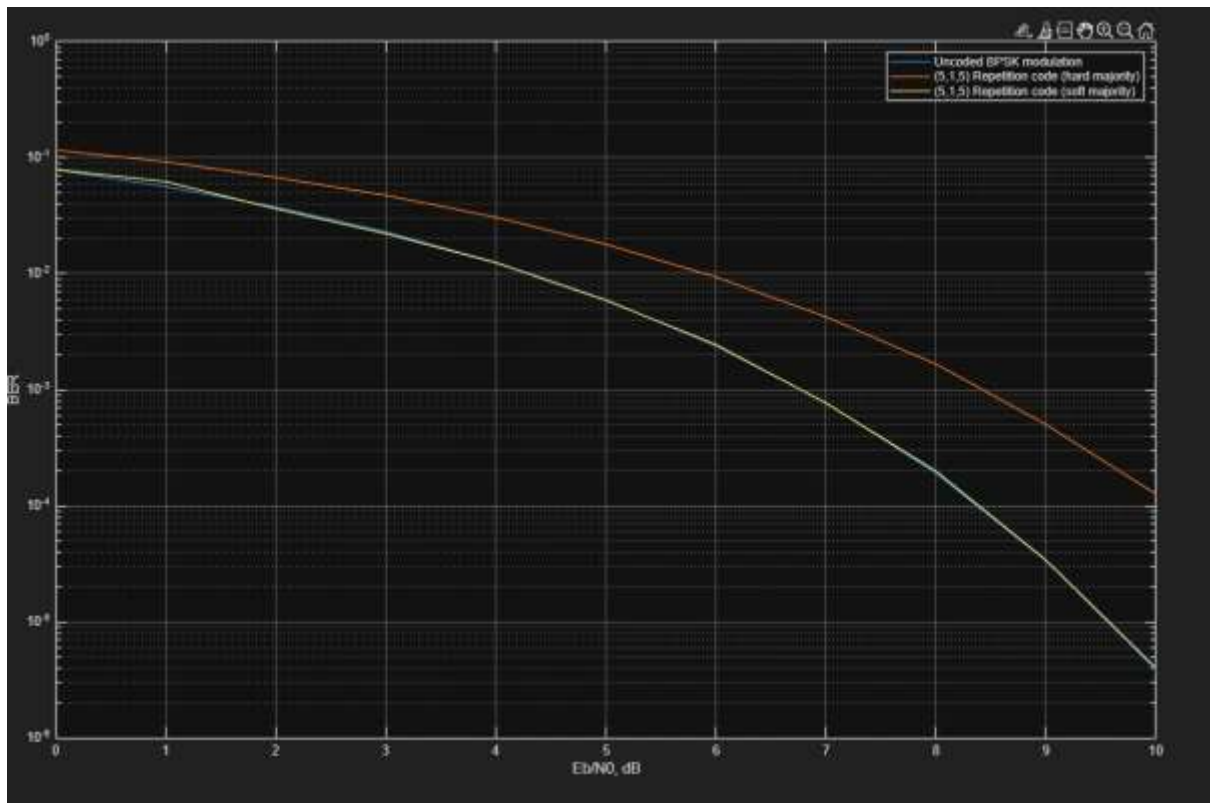


Figure 7 Soft majority decoding (5,1,5)

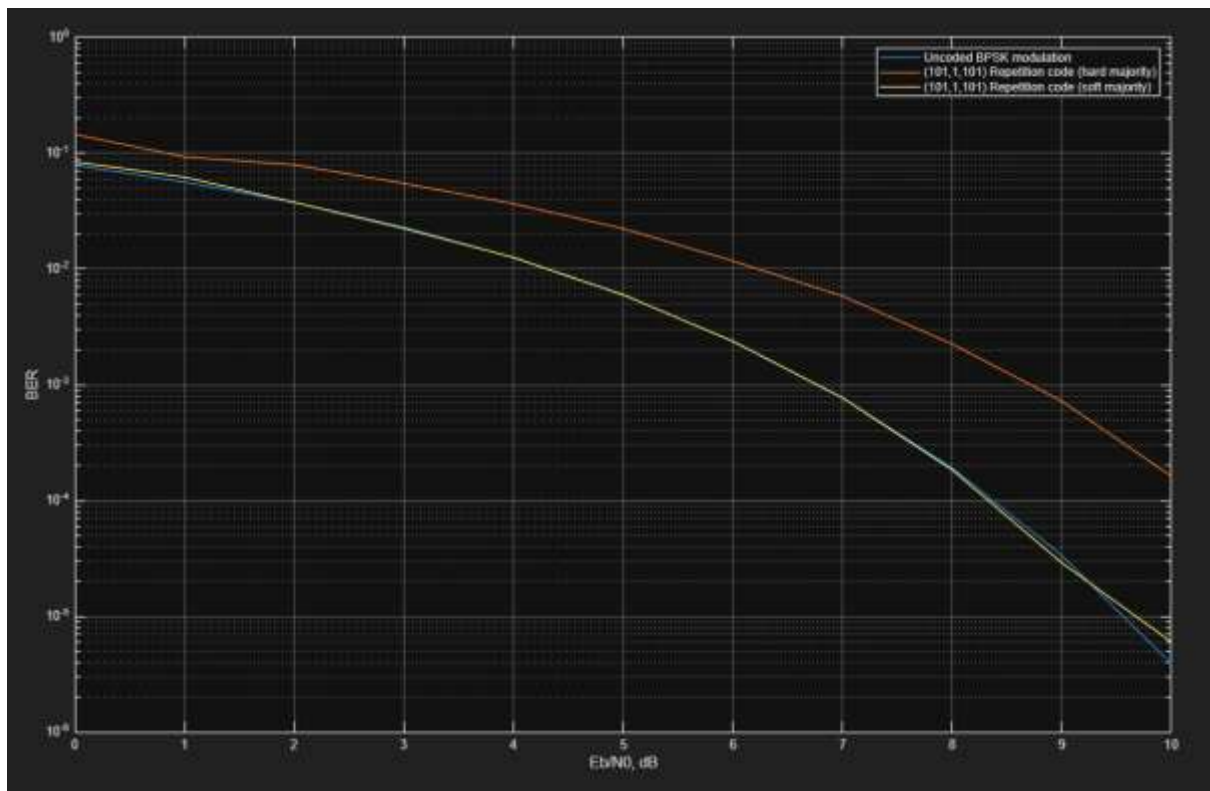


Figure 8: Soft majority decoding (101,1,101)

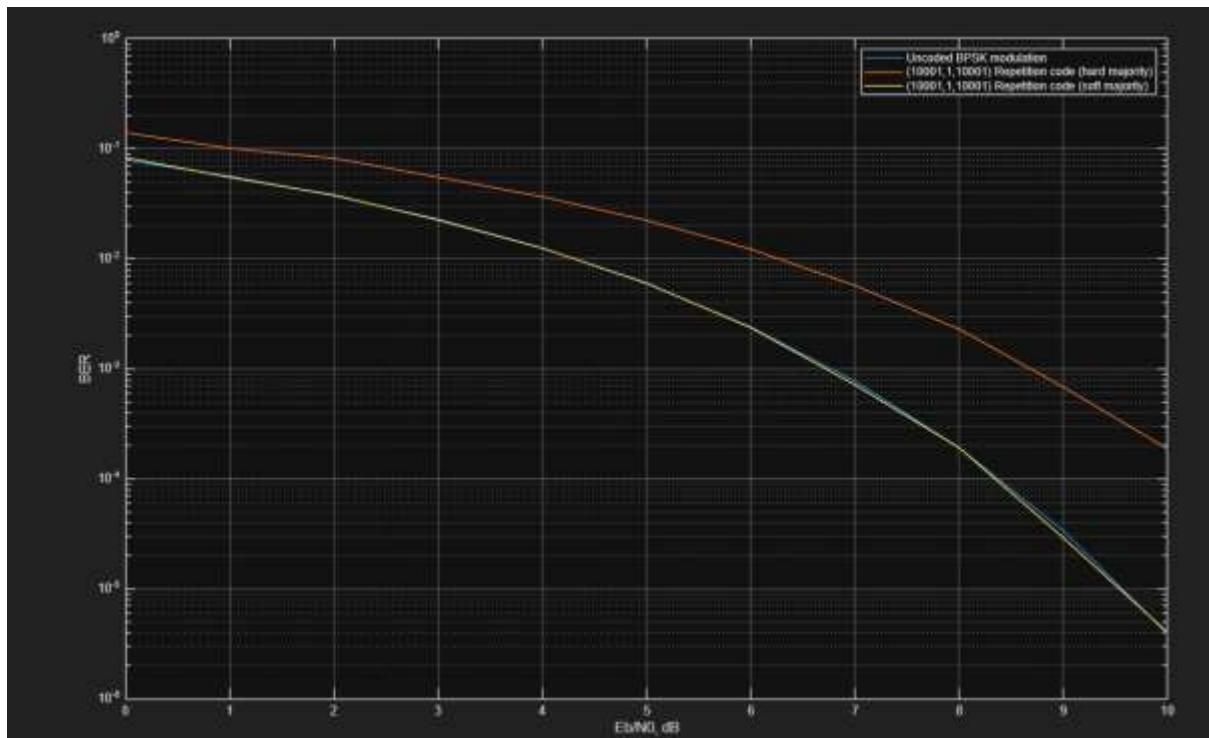


Figure 9: Soft majority decoding (10001,1,10001)

The (7,4,3) Hamming code

```

1 % Hamming(7,4) coding
2
3 clear('a')
4 a=[];
5
6 %-d; Message length
7 %-7; Codeword length
8 %-3; Minimum Hamming distance
9
10 %-m; Error rate
11 message=rand(1,N);
12 codeword=rand(1,N);
13
14 %-N; Number of codewords generated
15 N=1000; %Number of codewords generated
16
17 %-m; Message length
18 %-7; Codeword length
19 %-3; Minimum Hamming distance
20
21 %-m; Message length
22 %-7; Codeword length
23 %-3; Minimum Hamming distance
24
25 %-m; Message length
26 %-7; Codeword length
27 %-3; Minimum Hamming distance
28
29 %-m; Message length
30 %-7; Codeword length
31 %-3; Minimum Hamming distance
32
33 %-m; Message length
34 %-7; Codeword length
35 %-3; Minimum Hamming distance
36
37 %-m; Message length
38 %-7; Codeword length
39 %-3; Minimum Hamming distance
40
41 %-m; Message length
42 %-7; Codeword length
43 %-3; Minimum Hamming distance
44
45 %-m; Message length
46 %-7; Codeword length
47 %-3; Minimum Hamming distance
48
49 %-m; Message length
50 %-7; Codeword length
51 %-3; Minimum Hamming distance
52
53 %-m; Message length
54 %-7; Codeword length
55 %-3; Minimum Hamming distance
56
57 %-m; Message length
58 %-7; Codeword length
59 %-3; Minimum Hamming distance
60
61 %-m; Message length
62 %-7; Codeword length
63 %-3; Minimum Hamming distance
64
65 %-m; Message length
66 %-7; Codeword length
67 %-3; Minimum Hamming distance
68
69 %-m; Message length
70 %-7; Codeword length
71 %-3; Minimum Hamming distance
72
73 %-m; Message length
74 %-7; Codeword length
75 %-3; Minimum Hamming distance
76
77 %-m; Message length
78 %-7; Codeword length
79 %-3; Minimum Hamming distance
80
81 %-m; Message length
82 %-7; Codeword length
83 %-3; Minimum Hamming distance
84
85 %-m; Message length
86 %-7; Codeword length
87 %-3; Minimum Hamming distance
88
89 %-m; Message length
90 %-7; Codeword length
91 %-3; Minimum Hamming distance
92
93 %-m; Message length
94 %-7; Codeword length
95 %-3; Minimum Hamming distance
96
97 %-m; Message length
98 %-7; Codeword length
99 %-3; Minimum Hamming distance
100

```

```

47 %Count errors before decoding
48 num_err = sum(d ~= codeword);
49 if num_err == 0
50     err0 = err0 + 1;
51 elseif num_err == 1
52     err1 = err1 + 1;
53 else
54     err2p_count = err2p_count + 1;
55 end
56
57 error_pattern=zeros(1,N); %initialise error pattern vector to all zeros
58 syndromes=mod(H*d',2); %calculate syndromes
59
60 for i=1:N
61     a=isequal(syndromes',synd_table(i,:)); %compare syndrome
62     if a==1
63         error_pattern(i)=1;
64     end
65 end
66
67 decoded_message=mod(d+error_pattern,2); %correct received word
68
69 for i=1:K
70     if decoded_message(i)~=message(i)
71         errors=errors+1; %count bit errors in the data bits
72     end
73 end
74
75 end
76
77 ber(s)=errors/(frames(s)*K); %calculate BER
78
79
80 p0(s)=err0/frames(s); %errors for plotting
81 p1(s)=err1/frames(s);
82 p2p(s)=err2p_count/frames(s);
83
84
85 end
86
87 figure(1)
88 ber_uncoded=0.5*erfc(sqrt(Eb_N0));
89 semilogy(snr_uncoded,ber_uncoded,snr,ber);
90 legend1=sprintf('Uncoded BPSK modulation');
91 legend2=sprintf('(%d,%d,%d) Hamming code',N,K,D);
92 legend(legend1,legend2);
93 xlabel('Eb/N0, dB');
94 ylabel('BER');
95 grid on
96
97 %Error distribution per codeword
98 figure(2)
99 semilogy(snr,p0,'-o',snr,p1,'-s',snr,p2p,'-^');
100 legend('0 errors','1 error (correctable)','2 errors (uncorrectable)');
101 xlabel('Eb/N0, dB');
102 ylabel('Probability');
103 title('Error distribution per 7-bit codeword (before decoding)');

```

Task 3:

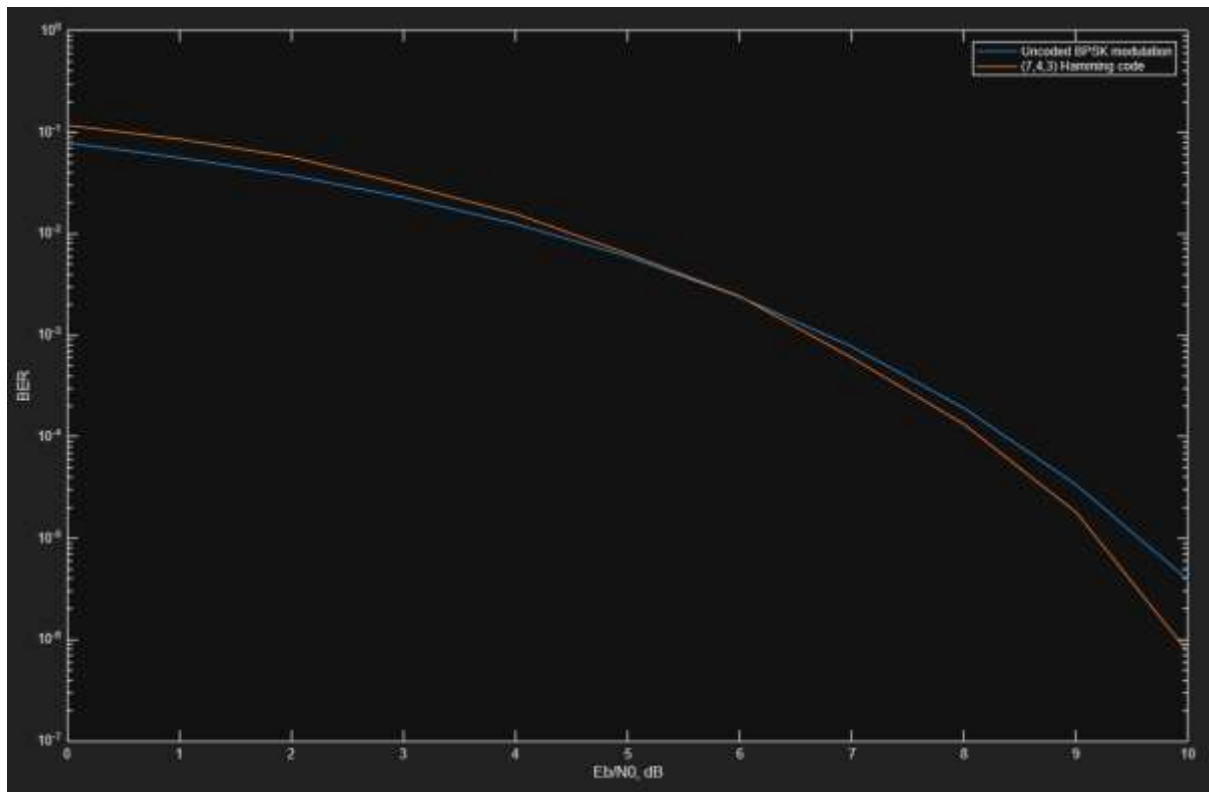


Figure 10: (7,4,3) Hamming code

Task 4:

Figure 10 shows that the Hamming code is a little improvement to the uncoded BPSK modulation. When the SNR is around 6dB, the Hamming code will start to get a lower BER than the unmodulated code.

The Hamming code uses a generator matrix to encode a 4-bit message, by making a 7-bit codeword with three parity bits and four message bits. It does this by using modulo 2 arithmetic between the generator matrix and message. Using a syndrome table, the hamming code can then correct faulty bits. The Hamming code can only correct 1 error in the 7-bit codeword so therefore the error correction capabilities are weak although because it has a better code rate than repetition codes, the energy per transmitted bit will be greater than the repetition code.

Error distribution:

The probability is calculated by the number of codewords with a given error count/total number of codewords. Figure 11 shows that at high noises, there is an even likelihood of no errors to two errors. As SNR increases, the likelihood of 0 errors increases until virtually perfect (probability of 1). The likelihood of 1 error decreases and 2 errors decrease more rapidly with more signal which is important as two errors means that the message will have been lost with no correction possible.

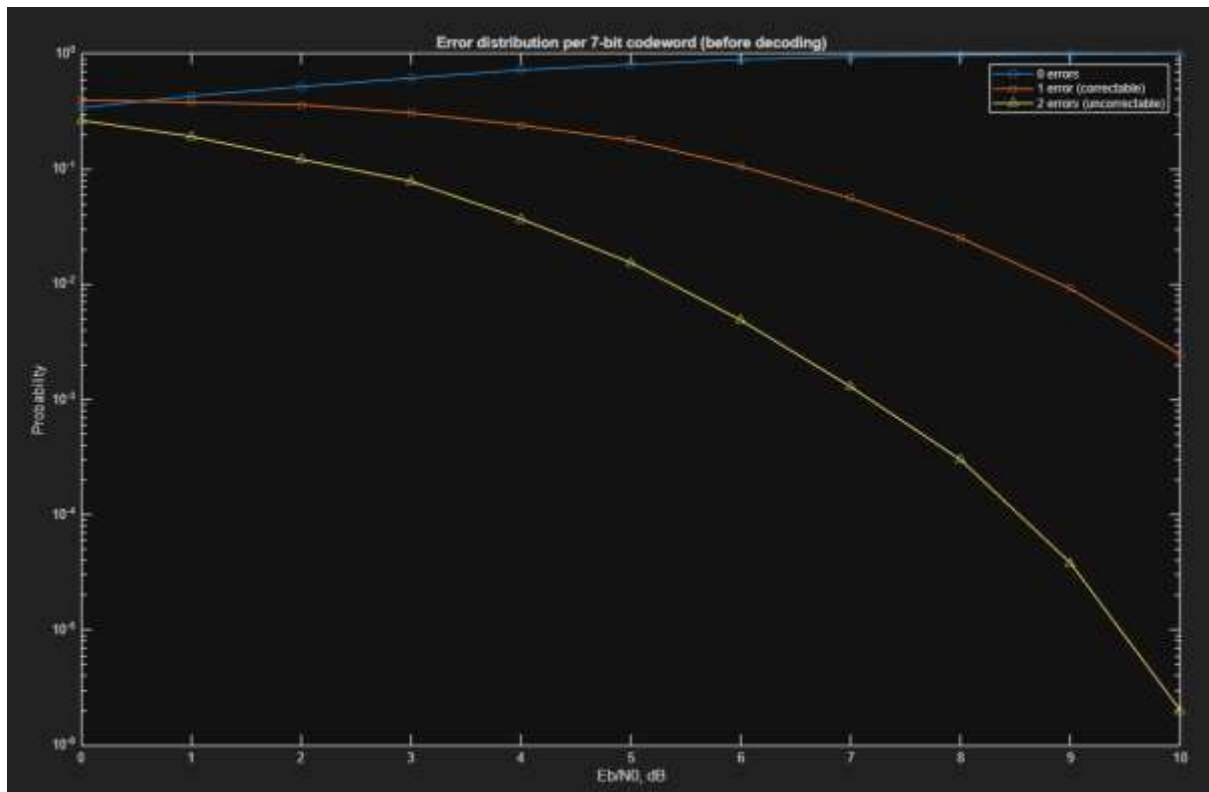


Figure 11: Error probability before decoding

Convolutional code

```

1 Experiment 1: Convolutional Code
2 clear();
3 clc();
4 K=6; % Code rate
5 const_length=7; %Constraint length of the convolutional code
6 g1=133; %First octal generator
7 g2=171; %Second octal generator
8 tb=const_length-1; %Terminating bits added to end of message (equal to number of memory elements) to force the encoder to contain all zero bits in the memory elements
9 K=1000-tb; %Message length (the added terminating bits do not contain any information)
10 n=(K+tb)/K; %Codeword length with terminating bits
11 message=zeros(1,K*ts);
12 codeword=zeros(1,n*(tb/K)); %Kb/Kc = number of terminating coded bits added to end of codeword
13 snr=[0 1 2 3 4 5 6 7 8 9 10];
14 frames=[10 100 100 100 100 1000 1000 10000 10000 10000]; %Number of codewords generated
15 snr_uncoded=10*log10(10,max(snr),length(snr));
16 Eb_N0=10*log10(snr/10);
17 ber_term=zeros(1,length(snr)); % Terminated trellis array
18 ber_trunc=zeros(1,length(snr)); % Non-zero truncated array
19 trellis=poly2trellis(const_length,[g1 g2]); %Matlab constructs a trellis based on the actual generators and constraint length of the convolutional code
20 for s=1:length(snr)
21     errors_term=0; % Terminated error counter
22     errors_trunc=0; % truncated error counter
23     sigma=sqrt(1/(2*Kc*tb_N0(s))); %Standard deviation of noise
24     for f=1:frames(s)
25         message=randi([0 1],1,K); %Generate random 0 and 1
26         for i=1:tb
27             message(K+i)=0; %Add terminating bits
28         end
29         codeword=convenc(message,trellis); %Convolutional encoder
30         x=2*codeword;
31         n=sigma*randn(1,length(x)); %Generate AWGN
32         y=x+n; %Add noise to BPSK signal
33         d=y/2;
34
35         decoded_message=vitdec(d,trellis,K*const_length,'term','hard'); %Terminated hard decision Viterbi decoding
36         for i=1:K
37             if decoded_message(i)~=message(i)
38                 errors_term=errors_term+1;
39             end
40         end
41         %Non-zero trellis state
42         message_trunc=randi([0 1],1,K); %Generate random 0 and 1. Terminating bits will not be added
43         codeword_trunc=convenc(message_trunc,trellis);
44         x_trunc=2*codeword_trunc;
45         n=sigma*randn(1,length(x_trunc));
46         y_trunc=x_trunc+n;
47         d_trunc=y_trunc/2;
48
49         decoded_trunc=vitdec(d_trunc,trellis,K*const_length,'trunc','hard'); %Truncated hard Viterbi decoding
50         for i=1:K
51             if decoded_trunc(i)~=message_trunc(i)
52                 errors_trunc=errors_trunc+1;
53             end
54         end
55     end
56 end

```

```

57
58     ber_term(s)=errors_term/(frames(s)*K);
59     ber_trunc(s)=errors_trunc/(frames(s)*K);
60 end
61 figure(1)
62 Eb_N0_uncoded=10.^(snr_uncoded/10);
63 ber_uncoded=0.5*erfc(sqrt(Eb_N0_uncoded));
64
65 semilogy(snr_uncoded,ber_uncoded, snr,ber_term, snr,ber_trunc);
66 grid on;
67
68 legend1=sprintf('Uncoded BPSK modulation');
69 legend2=sprintf('( %d,%d) terminated',g1,g2);
70 legend3=sprintf('( %d,%d) truncated',g1,g2);
71
72 legend(legend1,legend2,legend3);
73 xlabel('Eb/N0, dB');
74 ylabel('BER');
75

```

Task 5:

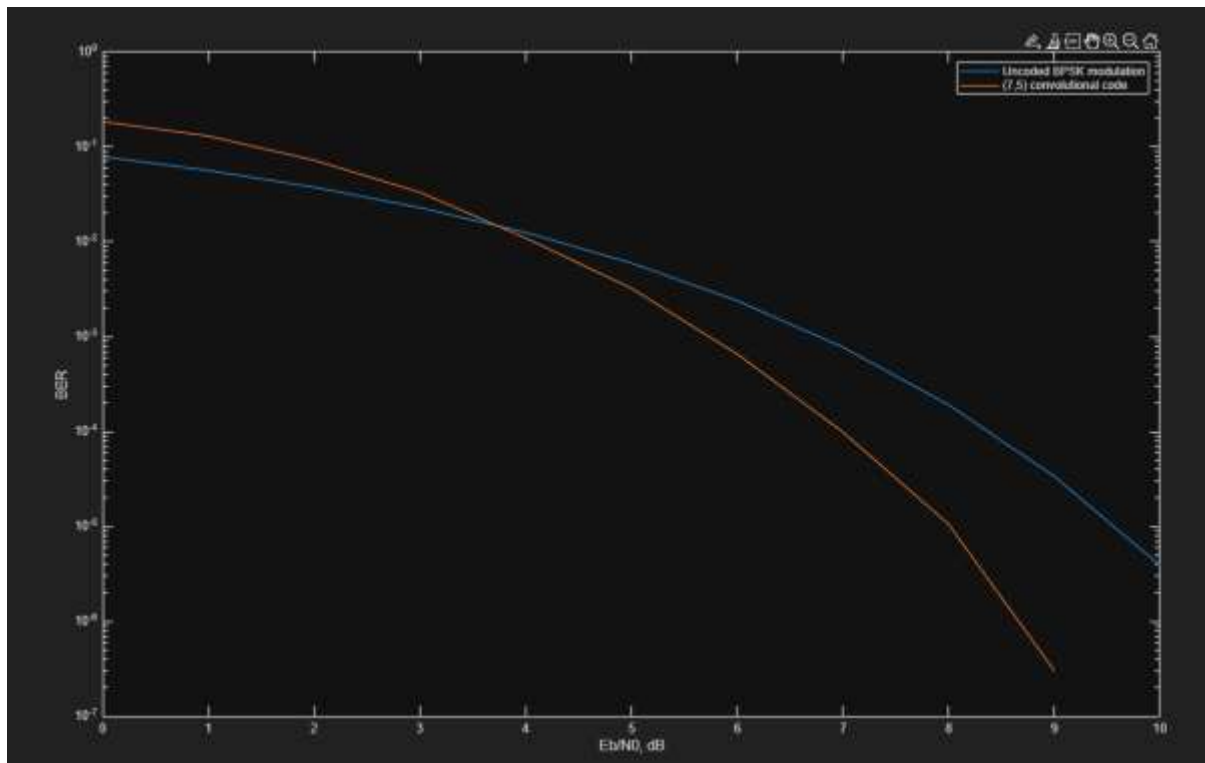


Figure 12: $(7,5)_8$ Convolution code (Constraint length 3)

In figure 12 after 4 dB, it is clear to see that convolutional codes have a much better performance than the uncoded BPSK modulation. At 9dB, the line stops indicating that the bit-error rate is so low that it can't be plotted on this graph. The code rate is 0.5 so has a lower code rate than the Hamming code which would imply there is more redundancy therefore less energy per message bit however the algorithm used for decoding can decode multiple bits at a time.

Task 6:

Figure 13 shows the convolutional code performing to a similar standard and this is because although the constraint length has increased, the complexity increases due to added memory elements. There are much more trellis paths that need to be sorted through for decoding.

Figure 14 shows a sweet spot between complexity and performance; it gets the best number of paths needed to find the best path in the trellis using the Viterbi decoding method. Constraint length 7 has better performance because the decoder essentially is given more memory allowing it to find the better path in the trellis for decoding.

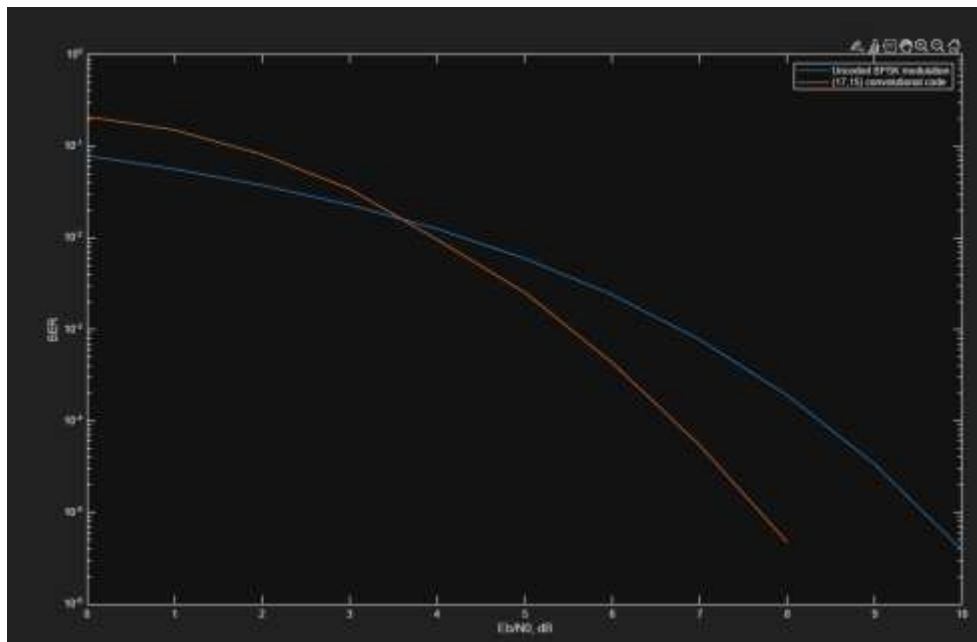


Figure 13: $(17,15)_8$ Convolution code (Constraint length 4)

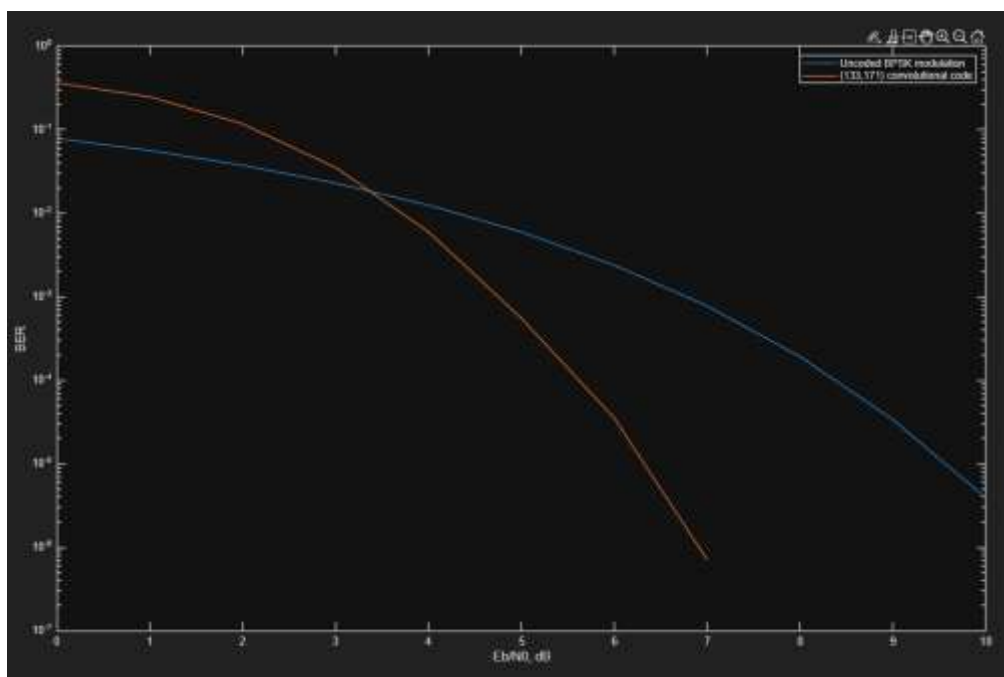


Figure 14: $(133,171)_8$ Convolution code (Constraint length 7)

Non-terminating trellis:

After decoding, the trellis returns to an all zero state by adding terminating bits equal to the number of memory elements in the encoder. By modifying the code so that the terminating bits are not added, the trellis algorithm has less information for the final branches and therefore fails to correct more bit errors per transmitted bit. Figure 15 shows how the performance weakens without terminating bits.

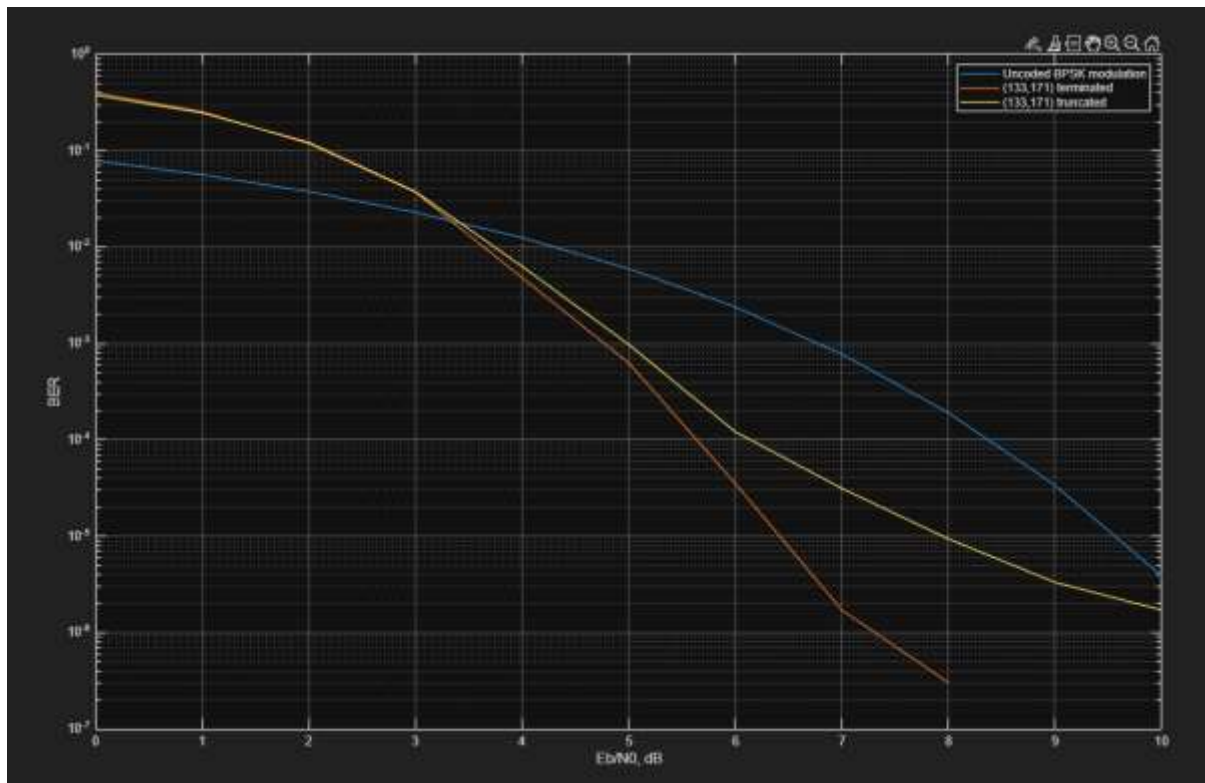


Figure 15: Terminating trellis performance vs non-zero state trellis

Task 7:

Repetition encoding has the worst performance with hamming code being slightly better and convolutional coding to be the best. Repetition codes redundancy is too high so code rate is very low. This causes lower energy per transmitted bit and therefore less likely to recover the full message. Hamming codes are more effective with less redundancy and a higher code rate. It's more efficient than repetition however still performs close to the uncoded BPSK modulation because the error correction abilities can only correct one error. Convolutional encoding is clearly the best because although it requires more memory elements and complexity, it can correct multiple bits at a time and can take a continuous input meaning it has the lowest BER across different SNR levels.